

CHAPTER 1: PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

Wireless networking has become an essential part of everyday digital life. Students use campus Wi-Fi, travelers rely on airport hotspots, employees connect to guest networks, and customers expect instant connectivity in cafés, hotels, hospitals, and shopping spaces. Although this convenience improves productivity and accessibility, it also exposes users to an environment in which not every visible network is trustworthy. A public wireless network can appear harmless while still being poorly secured, impersonated, or intentionally configured to capture data.

The Wi-Fi Security Analyzer project is proposed as a defensive and awareness-oriented solution for this problem. The central idea is not to attack networks or perform penetration activities, but to analyze the wireless landscape around a user and convert technical indicators into a meaningful security decision. The system aims to make risk visible before a user clicks the connect button.

The report has been prepared using the uploaded project format and guideline documents, the provided project code excerpt, and the project theory deck. The final narrative therefore reflects both the academic reporting structure expected by the university and the actual technical logic visible in the supplied implementation materials.

1.2 Motivation for the Work

The motivation behind this work comes from a gap between wireless security reality and user perception. Many users make connection decisions based on SSID familiarity, convenience, or signal strength, not on security. Attackers exploit exactly this behavior by creating open hotspots, misleading names, or Evil Twin access points that mimic legitimate networks.

Another motivation is usability. Existing wireless tools often present raw technical data such as dBm values, channel identifiers, and MAC addresses without interpreting what those values mean for a regular user. A beginner-friendly analyzer that explains why a network is risky can be more useful in day-to-day life than an expert-centric scanner that assumes deep networking knowledge.

1.3 Background and Technical Context

The slide deck describes the project as a Real-Time Wi-Fi Security Assessment and Risk Classification System. It frames the project around public Wi-Fi dangers such as eavesdropping, credential theft, malware injection, session hijacking, and Man-in-the-Middle attacks. It also positions the system as a layered security intelligence engine rather than a simple encryption checker.

The supplied source-code file supports this framing. The ``intelligent_risk_analysis()`` function applies a multi-parameter rule set instead of relying on a single network field. It first counts SSID repetitions, then evaluates each network using encryption-related rules, hidden SSID handling, duplicate SSID suspicion, and signal-based anomaly logic. The output includes not only a label but also a score, reasons, and user recommendations.

1.4 Problem Statement

Users frequently connect to nearby Wi-Fi networks without understanding the underlying security posture of those networks. Existing wireless scanning tools may identify basic encryption modes, but they often do not translate the data into understandable security guidance. As a result, users remain vulnerable to connecting to open, weakly protected, or impersonated access points.

The core problem addressed in this project is therefore the lack of an accessible, real-time, defensive tool that can classify nearby Wi-Fi networks into meaningful risk levels using multiple observable indicators and present those results in a clear interface.

1.5 Objective of the Work

The primary objective is to design and document a Wi-Fi analysis system that discovers nearby wireless networks, extracts useful security-related parameters, computes a rule-based risk score, and translates the outcome into plain-language risk classes and recommendations.

Secondary objectives include improving user awareness, supporting safer wireless choices in public locations, demonstrating a desktop-oriented prototype architecture, and documenting the project in a structured academic format aligned with university requirements.

1.6 Scope of the Project

The project scope includes wireless network discovery, parameter collection, risk scoring, graphical presentation of results, and report-oriented interpretation. It does not

include active exploitation, unauthorized access, password cracking, packet injection, or offensive security operations.

The present report is also careful about evidence boundaries. The supplied code excerpt clearly exposes the scoring logic and supporting imports, but it does not include every scanning or UI function. Therefore, the implementation chapter distinguishes between directly visible code evidence, slide-based design intent, and informed prototype interpretation.

1.7 Organization of the Project

Chapter 2 investigates related work in WLAN security, public hotspot threats, and existing approaches to risk detection. Chapter 3 presents the requirement artifacts of the project. Chapter 4 discusses the design methodology, architecture, and novelty of the proposed analyzer. Chapter 5 explains the prototype implementation and analytical validation based on the supplied code. Chapter 6 presents the outcomes and applicability of the system, while Chapter 7 concludes the report with limitations, recommendations, and future enhancement directions.

1.8 Summary

This chapter established the project context, motivation, objectives, and scope. It also clarified that the project aims to bridge wireless security analysis and user-friendly risk communication through an ethical, defensive, and educational approach.

Figure 1. Proposed Wi-Fi Security Analyzer Workflow

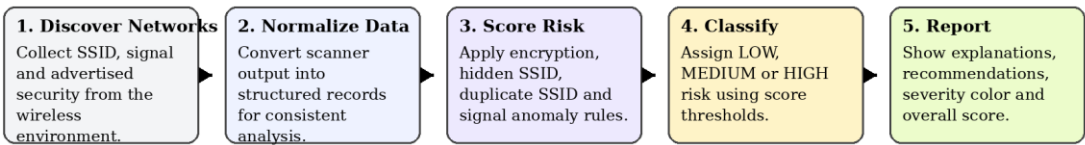


Figure 1.1 Proposed workflow of the Wi-Fi Security Analyzer

CHAPTER 2: RELATED WORK INVESTIGATION

2.1 Introduction

A good defensive Wi-Fi analyzer must be grounded in both protocol knowledge and threat intelligence. The related work for this project spans three broad areas: wireless security protocol comparison, rogue or Evil Twin access-point detection, and security usability tools that translate technical observations into human decisions.

The uploaded slide deck already points to literature on rogue AP detection and Wi-Fi protocol comparisons. To strengthen the academic framing, these materials are interpreted together with standard security guidance from NIST and recent discussion of rogue AP detection limits.

2.2 Core Area of the Project: WLAN Security and User-Centric Risk Assessment

Wireless LAN security is not determined only by whether a network advertises encryption. Network trustworthiness also depends on configuration quality, attack surface, identity consistency, and user behavior. A network using modern security settings can still be dangerous if it is spoofed, while a familiar SSID can be malicious if it is duplicated by an attacker in a crowded area.

For that reason, this project belongs to the intersection of wireless security assessment and cybersecurity usability. The proposed analyzer does not attempt deep protocol forensics. Instead, it collects observable signals and converts them into decision support for end users.

2.3 Existing Approaches / Methods

2.3.1 Encryption-only Security Checking

Many consumer tools treat encryption type as the main or only indicator of network safety. These tools flag open networks as unsafe and may distinguish between WEP, WPA2, and WPA3. This approach is useful as a first filter because protocol strength matters significantly. Historical guidance from NIST documents the evolution from WEP to WPA and WPA2, highlighting why older protections became inadequate.

However, encryption-only tools are incomplete. They do not explain why two networks with the same advertised protocol may pose different risks. They also do not help users understand identity-related attacks such as Evil Twin hotspots.

2.3.2 Rogue / Evil Twin Detection Research

The uploaded theory deck cites work on rogue access-point detection and Evil Twin prevention. Such work emphasizes that attackers can clone the identifiers of legitimate access points and lure users into connecting to a malicious AP. Once connected, users may be exposed to credential capture, traffic interception, or redirection.

Recent discussion in the literature further notes that wireless rogue AP attacks may bypass defenses built primarily for higher network layers. This is important because a user-facing analyzer cannot assume that enterprise monitoring systems will always detect local wireless impersonation.

2.3.3 Penetration-oriented or Expert-only Wireless Tools

A large number of wireless utilities are designed for auditing, packet capture, offensive testing, or expert diagnostics. They are powerful for specialists but often unsuitable for beginners. Their output is dense and technical, and it typically lacks human-readable recommendations.

The uploaded presentation explicitly differentiates the proposed system from such tools by describing it as a security intelligence layer. This distinction is significant from both a design and ethics perspective: the project focuses on awareness, interpretation, and safe decision-making rather than attack capability.

2.4 Pros and Cons of the Stated Approaches / Methods

Table 2.1 Comparison of related Wi-Fi security assessment approaches

Approach	Strengths	Limitations
Encryption-only check	Fast, easy to implement, useful first-level filter	Cannot detect spoofed identities or contextual anomalies
Rogue/Evil Twin detection research	Addresses real attacker behavior and identity deception	May require richer signals, infrastructure, or monitoring support
Expert wireless tools	Deep visibility for trained analysts	High complexity, low accessibility for everyday users
User-centric risk analyzer (proposed)	Readable output, actionable advice, ethical defensive use	Rule quality depends on observable features and chosen weights

2.5 Issues / Observations from Investigation

The investigation reveals a consistent problem: the best technical tools are often not the most usable, and the most usable tools are often too shallow. This project attempts to

reduce that gap by combining simple observable features with meaningful user communication.

A second observation is that no single metric is sufficient. Open networks deserve heavy penalty; WEP remains obsolete; WPA2 is acceptable but not ideal; and WPA3 offers a safer baseline. Even so, identity and environmental context still matter. Duplicate SSIDs, hidden SSIDs, and suspicious signal behavior can increase exposure independently of the cryptographic baseline.

A third observation is that academically sound system design should be honest about evidence. Because the supplied code excerpt centers on the risk engine rather than a complete end-to-end application, the report treats the prototype as a partially visible but conceptually coherent analyzer.

2.6 Summary

The related-work investigation supports the need for a layered, readable, and defensive Wi-Fi assessment tool. The proposed analyzer is justified by the limitations of encryption-only checks, the relevance of rogue AP threats, and the usability gap in many existing wireless tools.

Figure 2. Layered Risk Scoring Model

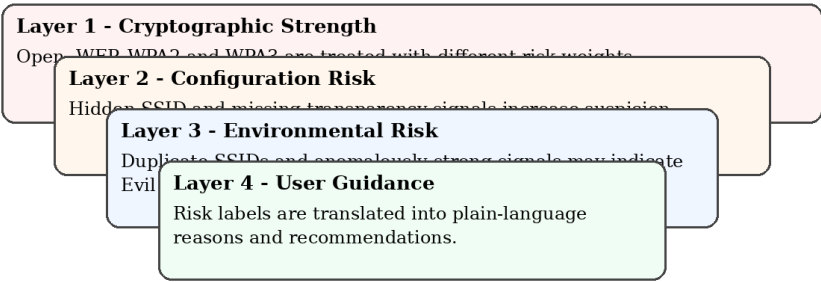


Figure 2.1 Layered model used to interpret wireless risk

CHAPTER 3: REQUIREMENT ARTIFACTS

3.1 Introduction

This chapter organizes the technical and non-technical requirements needed to realize the Wi-Fi Security Analyzer as a prototype system. The requirements are derived from the project theme, slide content, and supplied code structure.

Because the code excerpt is Python-based and imports PyQt5, Matplotlib, subprocess utilities, and optional ReportLab support, the requirements emphasize a desktop application capable of network analysis, visualization, and simple report generation.

3.2 Hardware and Software Requirements

Table 3.1 Hardware and software requirements of the prototype

Category	Requirement	Purpose
Hardware	Laptop/desktop with Wi-Fi adapter	Needed to detect nearby wireless networks
Hardware	At least 4 GB RAM (8 GB recommended)	Smooth GUI, charting, and analysis execution
Hardware	Standard display	To visualize tables, risk indicators, and recommendations
Software	Linux environment with wireless tools	Required by the presentation's NetworkManager-based scanning concept
Software	Python 3.x	Primary implementation language
Software	PyQt5	Desktop GUI framework
Software	Matplotlib	Charts and visual summaries
Software	ReportLab (optional)	PDF export support when available

3.3 Specific Project Requirements

3.3.1 Data Requirements

At minimum, the analyzer requires the SSID, a security descriptor, and a signal indication for each discovered network. The presentation also mentions BSSID and channel as useful parameters. The current code excerpt clearly assumes dictionary records containing keys such as `'ssid'`, `'security'`, and `'signal'`.

The data format should be normalized before analysis so that missing or malformed values do not break the risk engine. For example, the provided implementation safely handles missing SSIDs by substituting a hidden-network marker and guards signal parsing using a `'try/except'` block.

3.3.2 Functional Requirements

Table 3.2 Functional requirements of the Wi-Fi Security Analyzer

ID	Functional requirement
FR1	Discover nearby Wi-Fi networks using the host system's wireless interface
FR2	Capture observable attributes such as SSID, security type, and signal level
FR3	Compute a rule-based security score for each discovered network
FR4	Assign LOW, MEDIUM, or HIGH risk classes
FR5	Provide reasons and recommendations for each classification
FR6	Display findings through a user-friendly tabular dashboard
FR7	Provide an overall environment score summarizing the current wireless landscape
FR8	Support charting and optional report export

3.3.3 Performance and Security Requirements

The scoring engine should process discovered networks quickly enough for interactive use. The visible implementation performs one pass to count SSID occurrences and another pass to score networks, leading to linear complexity with respect to the number of discovered entries.

From a security standpoint, the tool must remain purely defensive. It should not transmit crafted traffic, attack access points, or collect user credentials. Its function is limited to scanning, interpreting, and educating.

3.3.4 Look and Feel Requirements

The system should provide a clean interface that highlights risk without overwhelming the user. The code imports Qt widgets such as table views, tabs, buttons, progress bars, combo boxes, file-dialog components, and message boxes. These imports imply an interface that can support multi-tab review, table-based summaries, progress-driven scan feedback, and export actions.

Color also plays a role in the visible prototype logic. The ``severity_color()`` helper maps HIGH to red, MEDIUM to dark yellow, and LOW to green, which supports immediate visual recognition.

3.3.5 Additional Constraints

Scanning support is platform-dependent because wireless network discovery relies on operating-system facilities. The slide deck explicitly references Linux and NetworkManager, and the code includes ``platform`` and ``subprocess`` imports, which

suggests OS-aware behavior. A full production version would therefore need platform abstraction and permissions management.

A second constraint is evidence completeness. The report documents only what is justified by the visible code and provided theory deck; therefore, all implementation claims remain within that boundary.

3.4 Summary

The requirement artifacts confirm that the system is feasible as a Python desktop prototype built around wireless scan normalization, rule-based risk analysis, readable output, and optional reporting support.

Figure 3. Prototype Software Architecture

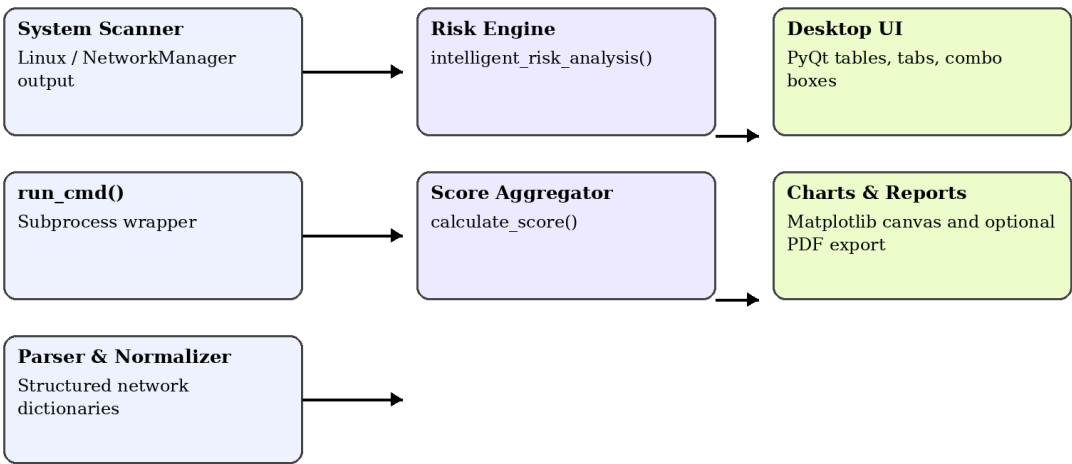


Figure 3.1 Prototype software architecture

CHAPTER 4: DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Methodology and Goal

The design methodology combines five steps: discovery, normalization, scoring, classification, and communication. First, networks are discovered using host-level wireless facilities. Second, raw scan results are normalized into structured Python records. Third, the risk engine assigns weights to specific observable indicators. Fourth, the final score is

translated into a LOW, MEDIUM, or HIGH class. Fifth, the result is communicated to the user with an explanation and a recommendation.

The goal is not maximum intrusion-detection depth; rather, it is practical decision support. The system should answer a human question: 'Should I trust this Wi-Fi network enough to connect to it?'

4.2 Functional Modules Design and Analysis

Table 4.1 Functional modules and their evidence base

Module	Description	Evidence base
Discovery module	Collects nearby networks from the host environment	Project deck design intent
Normalization module	Converts scan output into dictionaries with expected keys	Inferred from risk-engine input structure
Risk engine	Assigns scores, reasons, and recommendations	Directly visible in code
Environment scorer	Computes overall safety score from per-network classes	Directly visible in code
GUI module	Displays tables, controls, charts, and dialogs	Visible through imported PyQt5 and Matplotlib classes
Reporting module	Optional PDF/report generation	Visible through conditional ReportLab import

4.3 Risk Scoring Model

The project uses a deterministic rule-based scoring model. This is appropriate for an academic prototype because the logic is transparent, explainable, easy to validate, and easy to map to user-facing recommendations. Each network begins with a baseline score of zero and accumulates penalties based on observed traits.

The supplied code reveals the following weights: open network +50, WEP +40, WPA2 +20, WPA3 +5, hidden SSID +15, duplicate SSID +25, and unusually strong signal +15. After adding all relevant penalties, the system maps scores of 60 and above to HIGH risk, scores from 30 to 59 to MEDIUM risk, and scores below 30 to LOW risk.

Table 4.2 Rule-based risk scoring rubric used by the system

Observable condition	Risk contribution	Reason generated by system
Open network	+50	No authentication
WEP	+40	Easily crackable / obsolete security
WPA2	+20	Potential vulnerability to password-guessing attacks

WPA3	+5	Strong modern baseline
Hidden SSID	+15	Hidden SSID is not reliable protection
Duplicate SSID	+25	Possible Evil Twin or impersonation
Very strong signal (>85%)	+15	Possible fake hotspot / suspicious proximity

4.4 Software Architectural Design

Architecturally, the project is best represented as a lightweight desktop pipeline. System-level commands feed discovery data into a parser; normalized records flow into the risk engine; the analyzed output populates a table and optional charts; finally, the user may export or preserve the results.

The `run_cmd()` helper in the code is a small but important architectural decision. By isolating command execution into a reusable function, the system can adapt the discovery stage without rewriting the analysis logic.

4.5 Subsystem Services

The subsystems can be described as services: scanning service, scoring service, presentation service, and reporting service. The scoring service is the conceptual core because it converts technical data into explanations. The presentation service is equally important because the educational value of the tool depends on interface clarity.

The optional reporting service is suggested by the conditional `ReportLab` import. In a full implementation, this service could export scan snapshots or security summaries for record keeping, awareness training, or classroom demonstrations.

4.6 User Interface Design

The imported Qt widgets suggest a GUI organized around tabbed analysis views, scan action controls, tables, message boxes, progress indicators, and chart areas. The table component is suitable for listing discovered networks and their associated labels, while the chart component can present score summaries or class distributions.

The most important UI principle is interpretability. A user should not have to understand WLAN internals to act safely. Therefore, the interface should prioritize network name, security type, risk level, reasons, and recommendations ahead of lower-level technical metadata.

4.7 Design Novelty

The novelty of the project lies not in inventing a new wireless protocol, but in combining a defensive scan view with a readable scoring layer. The uploaded slides explicitly emphasize that the tool is not 'just encryption checking.' That design choice is important because it makes the analyzer closer to a decision engine than a passive scanner.

A second novelty is explainability. Every rule in the visible code produces not only a score impact but also a reason and a recommendation. This is a strong design choice for educational cybersecurity tools because it links observation, interpretation, and action.

4.8 Summary

The methodology chapter showed how the system transforms raw wireless observations into user-readable risk intelligence through transparent, rule-based logic. The design is lightweight, interpretable, and aligned with the defensive purpose of the project.

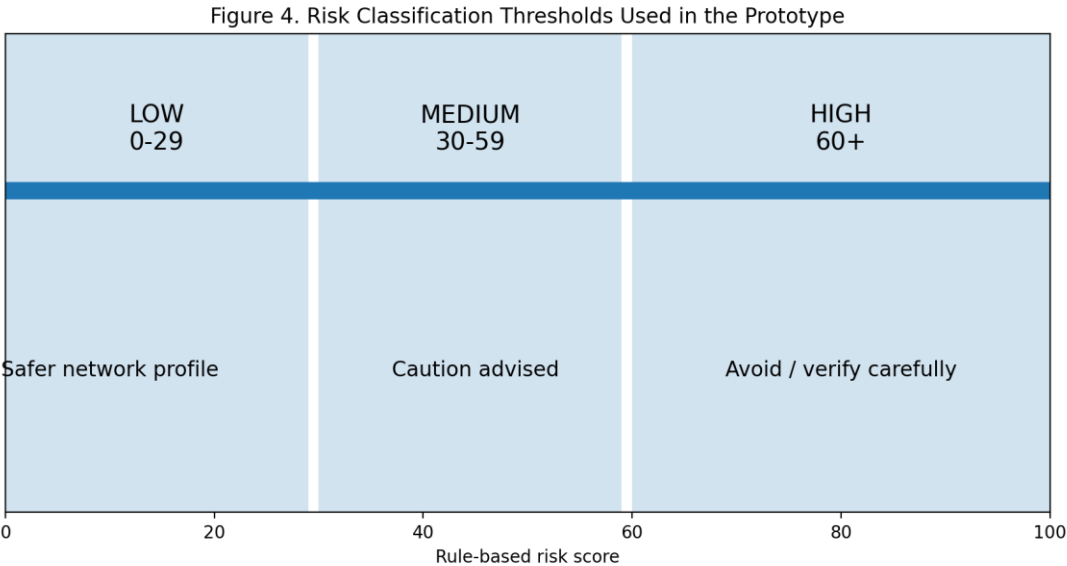


Figure 4.1 Risk-classification thresholds applied by the prototype

CHAPTER 5: TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 Outline

This chapter analyzes the technical implementation using the supplied Python code excerpt as the primary evidence source. Where the slide deck describes additional components not fully visible in code, those are described as intended or inferred prototype elements rather than confirmed completed features.

5.2 Technical Coding and Code Solutions

The provided code is Python-based and imports several libraries associated with a desktop analysis tool: PyQt5 for the interface, Matplotlib for embedded plotting, subprocess and platform utilities for system interaction, JSON and traceback for robustness, and ReportLab for optional PDF generation. Even though the visible file is relatively short, it reveals the architectural direction clearly.

The first major implementation element is the ``run_cmd()`` helper. It wraps ``subprocess.check_output()`` and returns decoded command output. This function is important because wireless discovery generally depends on OS-level commands or utilities. Abstracting command execution in one place simplifies later expansion and makes the rest of the pipeline cleaner.

The second and most important element is ``intelligent_risk_analysis(networks)``. The function first counts how many times each SSID appears. This preprocessing step supports later detection of duplicate SSIDs, which may indicate impersonation or an Evil Twin scenario. The function then iterates through each network and constructs a score, a reason list, and a recommendation list.

The scoring logic itself is explicit and explainable. Security strings are normalized to uppercase and checked against key categories: OPEN, WEP, WPA2, and WPA3. Hidden SSIDs increase the risk score because invisibility is not a guarantee of safety. Duplicate SSIDs add a substantial penalty because users may be tricked into connecting to a malicious access point that reuses a known network name. The code also examines signal values and assigns additional suspicion when the reported signal exceeds 85 percent, reflecting the idea that unexpectedly strong signals can indicate a nearby fake hotspot.

5.3 Working Layout of Forms / Dashboard

Although the full GUI code is not present in the supplied text file, the imported widget set indicates the intended layout. A typical dashboard for this project would include a scan button, a table of results, a severity legend, a summary score, and optional charts. The presence of ``QTabWidget`` suggests that the interface may separate live scan results, recommendations, charts, and export controls into different views.

The function ``severity_color()`` supports consistent visual encoding by linking HIGH, MEDIUM, and LOW values to intuitive colors. This is a useful design practice in security dashboards because users often form a first impression visually before reading explanatory text.

5.4 Prototype Submission / Working Pipeline

The working pipeline can be described in six stages: network scan, raw output capture, structured record creation, risk scoring, summary-score computation, and GUI/report rendering. The code confirms the two analysis stages explicitly: per-network classification and overall safety aggregation.

The helper `calculate_score(networks)` transforms per-network labels into a single environment score by assigning 90 points to LOW-risk networks, 60 points to MEDIUM-risk networks, and 30 points to HIGH-risk networks, then averaging across the list. This implementation does not measure cryptographic quality directly; instead, it compresses the observed wireless environment into a simple interpretable number.

Table 5.1 Mapping of visible code elements to prototype roles

Visible implementation element	Observed role in prototype
<code>run_cmd()</code>	Executes system-level commands and decodes scanner output
<code>intelligent_risk_analysis()</code>	Assigns score, risk class, reasons, and recommendations per network
<code>calculate_score()</code>	Produces a single environment score from per-network labels
<code>severity_color()</code>	Maps risk classes to dashboard colors
PyQt5 imports	Support desktop GUI widgets, progress bars, and tabbed layout
Matplotlib FigureCanvasQTAgg	Supports embedded charts in the interface
ReportLab optional import	Indicates possible PDF export/report generation

5.5 Test and Validation

Because the supplied evidence centers on the scoring engine, validation in this report is framed as rule-based functional validation rather than field-deployed benchmarking. The main test question is whether the engine behaves consistently and sensibly across representative network scenarios.

The visible rules support deterministic test cases. For example, an open network should reach HIGH risk even before other penalties are considered. A WPA3 network without additional suspicious indicators should remain LOW risk. A WPA2 network duplicated across the environment should move into a higher concern category because the duplicate-SSID penalty compounds the baseline score.

Table 5.2 Representative validation test scenarios

Scenario	Input traits	Expected score	Expected class
Campus secured Wi-Fi	WPA3, named SSID, normal signal	5	LOW
Home/office WPA2	WPA2, named SSID, normal signal	20	LOW
Hidden WPA2 network	WPA2 + hidden SSID	35	MEDIUM
Duplicate WPA2 network	WPA2 + duplicate SSID	45	MEDIUM
Open hotspot	OPEN only	50	MEDIUM / cautionary near-high
Open suspicious twin	OPEN + duplicate SSID	75	HIGH
WEP network	WEP only	40	MEDIUM

Figure 5.1 Representative validation test scenarios

SSID	BSSID	Security	Channel	Signal	Risk	Reason & Vulnerability	Recommendation
1 VITBPL	a4:c7:f6:fc:a2:51	WPA2-Enterprise	58 (22 %)	48%	LOW	WPA2 → vulnerable to dictionary attacks	Use WPA3 + strong password
2 Aman's realme P1 5G	5a:b9:8c:93:08:c7	WPA3-Personal	1	85%	LOW	WPA3 → strong security	Keep using WPA3
3 realme 8	3a:90:f9:af:66:3e	WPA2-Personal	6	97%	MEDIUM	WPA2 → vulnerable to dictionary attacks Very strong ...	Use WPA3 + strong password Check for fake hotspot

Figure 5.2 Dashboard representing Security Score and Network Risk Distribution



5.6 Performance Analysis (Graphs / Charts)

The computational complexity of the risk engine is linear in the number of discovered networks. One pass computes SSID frequencies and another pass assigns scores, making the logic efficient for small to medium wireless environments such as classrooms, offices, cafés, or airport waiting areas.

Performance in this context is not limited to execution time. Interpretability is also a performance criterion. A fast system that produces cryptic output is less valuable than a slightly richer system that explains itself. The present prototype performs well in this respect because reasons and recommendations are generated alongside the risk label.

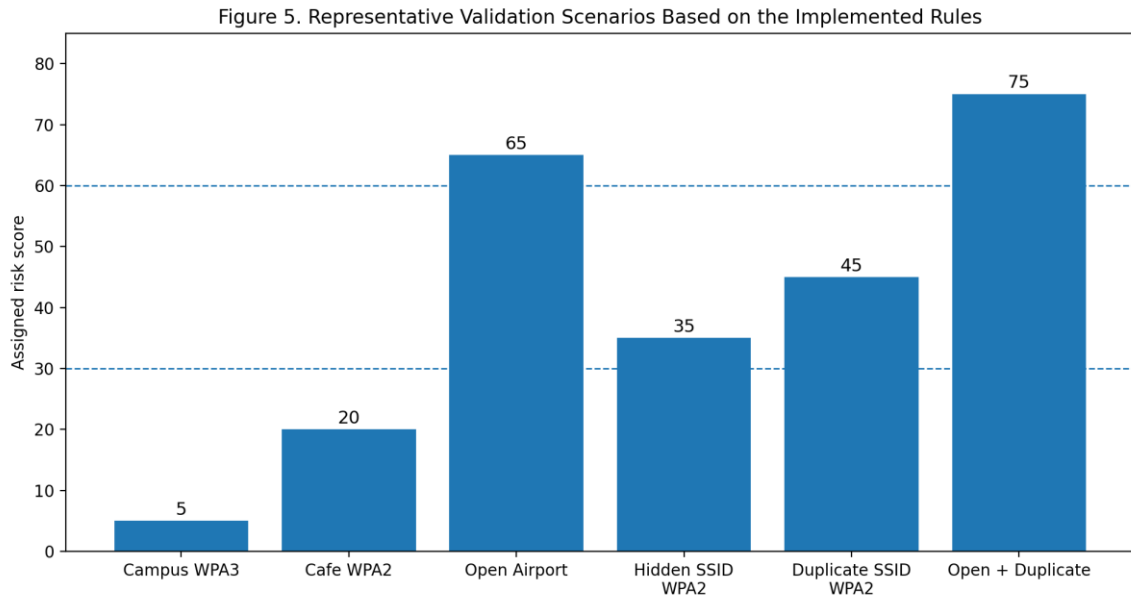


Figure 5.3 Representative validation scenarios

5.7 Security and Ethical Analysis

The project is ethically positioned as a defensive security aid. The slide deck clearly describes the approach as awareness-based and excludes unauthorized access attempts. This distinction is critical in cybersecurity education because it demonstrates how analysis tools can improve safety without enabling abuse.

A practical ethical benefit of the current design is that the analyzer requires only observational scan data and local reasoning. It does not need to attack or authenticate to target networks. This reduces both legal risk and operational risk in classroom and demonstration environments.

5.8 Summary

The technical implementation chapter shows that the supplied code already contains the essential intelligence layer of the system. The prototype is lightweight but meaningful: it converts raw wireless observations into scores, labels, explanations, recommendations, and an overall environment indicator.

Figure 6. User Decision Support Flow

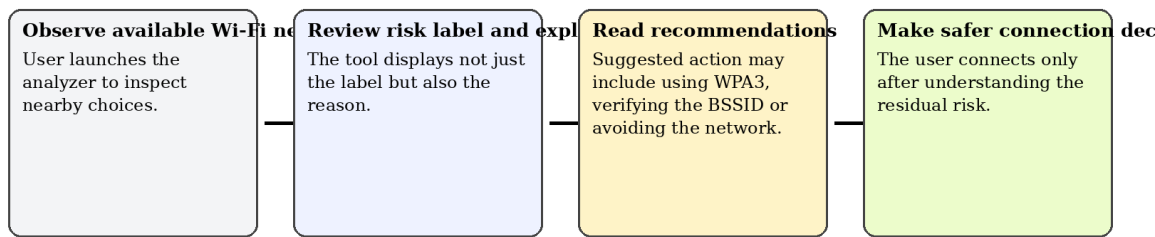


Figure 5.4 User decision-support flow enabled by the analyzer

CHAPTER 6: PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter presents the practical outcomes of the project and explains where the Wi-Fi Security Analyzer can be useful in real environments.

6.2 Key Implementation Outlines of the System

The project outcome is a documented prototype architecture for real-time Wi-Fi risk assessment. The most valuable deliverable is the scoring model that bridges technical wireless indicators and understandable security advice.

Another important outcome is the educational framing of the tool. By generating reasons such as 'Open network - no authentication' or 'Duplicate SSID - possible Evil Twin,' the analyzer helps users learn why a network is risky rather than merely warning them.

6.3 Significant Project Outcomes

Table 6.1 Significant project outcomes and their practical significance

Outcome	Practical significance
Readable risk classes	Users can immediately interpret wireless safety level
Actionable recommendations	Encourages safer behavior before connection
Multi-parameter scoring	Improves realism beyond encryption-only checks
Desktop-friendly prototype	Suitable for demos, labs, and student projects

Expandable architecture	Can be extended with more context signals in future versions
-------------------------	--

6.4 Project Applicability in Real-world Applications

The analyzer is directly applicable in educational institutions, public hotspots, travel environments, guest office networks, and cybersecurity awareness workshops. In these spaces, the problem is not a lack of network availability but a lack of trustworthy interpretation.

The project is also useful for beginner training. Students who are new to wireless security can observe how different combinations of protocol type, identity duplication, and signal behavior affect overall trust decisions. This makes the project relevant as a teaching aid in cybersecurity and digital-forensics curricula.

With additional engineering, the same concept could support enterprise guest-network guidance, hotel Wi-Fi safety kiosks, campus awareness campaigns, or security training modules for frequent travelers.

6.5 Inference

The principal inference is that the project succeeds best as an explainable security assistant. Its strongest contribution is not raw scan depth but translation of wireless observations into defensible connection decisions.

CHAPTER 7: CONCLUSIONS AND RECOMMENDATION

7.1 Outline

This chapter concludes the report, summarizes the system limitations, and recommends future enhancements.

7.2 Limitations / Constraints of the System

The prototype is constrained by observable scan features. It can classify based on advertised security, identity repetition, and signal anomalies, but it cannot fully validate whether a network is genuinely legitimate without richer infrastructure, historical baselines, or deeper protocol inspection.

A second limitation is platform dependence. Wireless scanning often differs across operating systems, driver stacks, and permissions. The project deck indicates a Linux-and-NetworkManager orientation, which means portability requires additional work.

A third limitation is evidence scope. The supplied code excerpt exposes the intelligence layer but not the complete production application. Therefore, the present report documents a prototype with strong design evidence rather than claiming an industrially complete software product.

7.3 Future Enhancements

Future versions can include BSSID consistency analysis, channel correlation, historical profile comparison, time-based anomaly tracking, and exportable incident logs. These improvements would strengthen rogue AP detection without changing the ethical defensive purpose of the tool.

Machine-learning extensions may also be considered, but only after a sufficiently rich and well-labeled dataset is available. For the current prototype stage, the rule-based approach remains a better choice because it is transparent and easier to justify academically.

Another future enhancement is cross-platform support with modular scanner backends. A production release could support Linux, Windows, and macOS through adapter-specific discovery modules while keeping the same scoring interface.

7.4 Recommendation

It is recommended that future student teams continue the project as a defensive educational tool, preserving its explainability while expanding the evidence available to the risk engine. The current design should not be converted into an offensive scanner because its value lies in awareness, responsible security behavior, and decision support.

For academic demonstrations, the project should be showcased with representative scenarios showing how the same SSID may vary in trust depending on duplication, visibility, and protocol settings. This communicates the central insight of the system very effectively.

7.5 Inference

In conclusion, the Wi-Fi Security Analyzer is a meaningful cybersecurity prototype that addresses a real everyday problem: users often do not know which Wi-Fi networks are safe enough to trust. By combining transparent rule-based scoring with understandable explanations and recommendations, the project transforms wireless scan data into actionable security intelligence.

REFERENCES

- Rogue / Evil Twin Detection
- https://www.ijert.org/detection-of-rogue-access-point-to-prevent-evil-twin-attack-in-wireless-network?utm_
- Modern Evil Twin Detection
- <https://arxiv.org/abs/2512.10470>
- Wireless Security Protocol Comparison (WEP vs WPA vs WPA2 vs RSN)
- <https://eudl.eu/doi/10.4108/e-forensics.2008.2654>
- Comparative Study of WiFi Security Protocols (WEP, WPA, WPA2)
- <https://airccse.org/journal/ijans/papers/4314ijans01.pdf>

APPENDIX A: CORE RULES VISIBLE IN THE SUPPLIED CODE

The following facts are directly visible in the supplied source-code excerpt and were used to ground this report:

- The implementation language is Python.
- A desktop user interface is implied through PyQt5 widget imports.
- Matplotlib integration is planned via `FigureCanvasQTAgg`.
- Optional PDF/report export is implied by the conditional ReportLab import.
- The central scoring function is `intelligent\_risk\_analysis(networks)`.
- Duplicate SSIDs are identified using a first-pass SSID frequency count.
- Risk thresholds are: HIGH ≥ 60 , MEDIUM ≥ 30 , LOW < 30 .
- An environment-level score is computed by `calculate\_score(networks)`.

APPENDIX B: SAMPLE PSEUDOCODE FOR NETWORK CLASSIFICATION

```
Algorithm ClassifyNetworks(networks):  
1. Count frequency of each SSID.  
2. For each network n in networks:  
3.     score <- 0  
4.     if n.security is OPEN then score += 50  
5.     else if WEP then score += 40  
6.     else if WPA2 then score += 20  
7.     else if WPA3 then score += 5  
8.     if SSID missing then score += 15  
9.     if duplicate SSID then score += 25  
10.    if signal > 85% then score += 15  
11.    if score  $\geq 60$  label HIGH  
12.    else if score  $\geq 30$  label MEDIUM  
13.    else label LOW  
14. Return labeled network list.
```